

Discrete-Event Simulation and Constrained Optimisation of Emergency-Department Patient Flow*

A From-Scratch Replication of Lv et al. (2026) and Four Extensions

Sudip Kumar Saha
Student ID 2105152
Dhaka, Bangladesh
2105152@student.cse.buet.ac.bd

Ahnaf Akib Tanim
Student ID 2105154
Dhaka, Bangladesh
2105154@student.cse.buet.ac.bd

Md. Shahjalal Rumman
Student ID 2105165
Dhaka, Bangladesh
2105165@student.cse.buet.ac.bd

Md. Dulal Hossain
Student ID 2105169
Dhaka, Bangladesh
2105169@student.cse.buet.ac.bd

Md. Shoaib Hossain
Student ID 2105170
Dhaka, Bangladesh
2105170@student.cse.buet.ac.bd

Abstract

Emergency departments (EDs) must decide, every time a physician becomes free, which of three competing queues to serve: newly arrived urgent patients, newly arrived non-urgent patients, or patients returning for a follow-up after diagnostics. We rebuild from scratch the discrete-event simulation (DES) of Lv et al. (2026), which compares three physician-scheduling heuristics on a Chinese tertiary-hospital ED, and validate it against every published KPI. We then extend it in four directions the base paper does not attempt. First, we add the $c\mu$ rule and its generalisation, scheduling policies that are *provably* optimal for a stated delay-cost objective rather than hand-designed. Second, we replace the paper’s two-stage brute-force grid search over the slack thresholds (k_1, k_2) with a hand-implemented Nelder–Mead simplex driven by a sample-average approximation and an exterior penalty for the service-level constraint. Third, we introduce an explicit weekly cost model and jointly optimise the integer physician roster together with the thresholds. Fourth, we apply exact common random numbers (CRN) and quantify the resulting variance reduction. The rebuild reproduces all six structural findings of the paper and its equipment-utilisation table to within 0.5 percentage points, although absolute waiting times run 12–25% higher at the implied system load $\rho \approx 0.87$. Nelder–Mead matches the lattice optimum to 0.006 minutes using $19\times$ fewer simulations; the cost-optimal roster saves 20.7% of weekly cost; CRN removes 98.9% of comparison variance. Notably, the paper’s own published thresholds are infeasible on our model, which motivates constrained re-tuning rather than parameter reuse.

CCS Concepts

• **Applied computing** → **Health informatics**; • **Computing methodologies** → **Discrete-event simulation**; • **Theory of computation** → **Scheduling algorithms**; • **Mathematics of computing** → *Nonconvex optimization*.

Keywords

discrete-event simulation, emergency department, patient scheduling, $c\mu$ rule, Nelder–Mead, simulation optimisation, common random numbers, variance reduction, staffing

1 Introduction and Base Paper Review

1.1 Background

Emergency departments are the classic congestion bottleneck of a hospital. Demand is stochastic and strongly non-stationary, service is heterogeneous, and the critical resource — physician time — is both scarce and expensive. Because patients are triaged into urgency levels with different clinical waiting targets, an ED is naturally modelled as a *multi-class queue with deadlines*. Simulation is the standard tool for evaluating operational policies in this setting [16, 18], because the combination of time-varying arrival rates, non-exponential service and history-dependent priority rules puts exact analytic treatment out of reach [5].

1.2 The base paper

Our base paper is Lv, Liu, Yan and Wang [10]. It constructs a DES of a Grade-A tertiary-hospital ED in China restricted to triage Level III (urgent) and Level IV (non-urgent) patients, who together account for the great majority of ED visits. Each patient arrives according to a non-homogeneous Poisson process whose rate $\lambda_{d,h}$ varies by day and hour, waits for an *initial* consultation, may then require diagnostics (laboratory, X-ray, CT or B-ultrasound), and if so returns to compete for the same physicians in a *follow-up* queue. Physicians work three shifts per day staffed (5, 5, 3).

The paper’s contribution is to compare three physician-scheduling policies within one unified simulation framework:

- **Initial-First (IFP)**: new arrivals always take precedence, Level III before Level IV, and follow-ups are served only with leftover capacity.
- **Alternating 1:1 (ALT)**: at most half the physicians may work on initial consultations at any time; the remainder take follow-ups.
- **Slack-Based (SBP)**: a patient of level ℓ is flagged *urgent* once their waiting time reaches $T_\ell - k_\ell$, where T_ℓ is the triage target and k_ℓ is a tunable slack parameter [3]. Priority

*CSE 402 — Numerical Analysis, Simulation and Modeling Sessional. **Group 02**, **Section C**. Code and all generated results: <https://github.com/Ahnaf-Akib-Tanim/Numeric-Project-submission>

Table 1: Gaps in the base paper and the corresponding contribution of this work.

Paper does	Paper does not	Our work
Grid-search two thresholds	Optimise the <i>staffing level</i> jointly	\$2.7
Compare three heuristics	Compare against a provably optimal rule	\$2.5
Brute-force search	Use a derivative-free numerical optimiser	\$2.6
100 independent replications	Apply variance reduction	\$2.8
Report results	Release engine or data	\$3

is then urgent-L3 > urgent-L4 > follow-up > non-urgent initial.

SBP’s thresholds are tuned by a two-stage brute-force grid search (coarse step 1.0, then fine step 0.1), yielding $(k_1, k_2) = (13.1, 2.1)$ and a reported reduction in mean waiting time from 46.26 min (IFP) to 35.25 min, i.e. 23.8%, with no service-level violations. Statistical significance is established with paired t -tests, a Bonferroni correction and Cohen’s d .

1.3 Gaps and objectives

The base paper is a well-constructed comparison, but it leaves four concrete openings, summarised in Table 1. Neither its source code nor its operational dataset is public; only summary statistics are published, which is nevertheless sufficient to rebuild an equivalent simulator.

Accordingly the objectives of this work are: (i) to rebuild the model from scratch and establish how faithfully the published results can be reproduced from the published parameters alone; (ii) to introduce a scheduling rule derived from queueing theory rather than designed by hand; (iii) to replace brute-force tuning with a genuine numerical optimiser under an explicit constraint, and to measure the efficiency gain in a hardware-independent way; (iv) to answer the staffing question the paper only probes by sensitivity analysis; and (v) to sharpen the statistical comparison through variance reduction.

2 Methodology

2.1 System model

Let t denote simulated time in minutes over a horizon of one operational week ($T = 10,080$). Following the paper, the day and hour indices are

$$d = \lfloor \frac{t}{1440} \rfloor \bmod 7, \quad h = \lfloor \frac{t \bmod 1440}{60} \rfloor, \quad (1)$$

so the arrival intensity $\lambda_{d,h}$ is piecewise constant with a weekly cycle. Each arrival is assigned triage level $\ell \in \{3, 4\}$ with probabilities 0.25 and 0.75 and inherits the clinical waiting target $T_3 = 30$, $T_4 = 120$ minutes. The state is three FIFO queues, $W_{\text{init},3}^t$, $W_{\text{init},4}^t$ and W_{follow}^t , competing for R^t physicians.

Algorithm 1 Discrete-event main loop

```

1: push all arrivals and shift changes onto the heap  $H$ 
2: while  $H \neq \emptyset$  do
3:    $e \leftarrow \text{POP}_{\text{MIN}}(H)$ 
4:   if  $e.\text{time} \geq T$  then break
5:   end if
6:    $\text{now} \leftarrow e.\text{time}$ ; accrue utilisation integrals
7:   apply the state transition of  $e$ 
8:    $R \leftarrow \text{FREEPHYSICIANS}(\text{now})$ 
9:   if  $R > 0$  and some queue is non-empty then
10:     $P \leftarrow \text{POLICY.ALLOCATE}(\text{state}, \text{now}, R)$ 
11:    start service for every patient in  $P$ 
12:   end if
13: end while

```

2.2 Discrete-event engine

The simulator is a hand-written event loop over a binary min-heap; no process-oriented simulation library is used, so every mechanism described in the paper is explicit in the code. Events are 4-tuples ($\text{time}, \text{type}, \text{seq}, \text{payload}$) ordered lexicographically; the monotone counter seq makes the ordering total and therefore the replication bit-for-bit reproducible. Five event types are processed: shift change, arrival, consultation end, examination device release and examination results ready. After every event a single DISPATCH routine is invoked, and *that* is the only point at which the scheduling policy acts (Algorithm 1). Swapping a policy therefore changes no other line of the simulator, which makes the comparison structurally fair.

Service is non-preemptive: when a shift shrinks at 22:00 no consultation is interrupted; the roster simply blocks new starts until the busy count falls back below capacity.

2.3 Arrival generation by thinning

Because $\lambda(t)$ is a 168-piece step function, inverting its cumulative intensity is error-prone. We instead sample the non-homogeneous Poisson process by Lewis–Shedler thinning [9]: with $\lambda^* = \max_{d,h} \lambda_{d,h} = 23.05/\text{h}$, generate a homogeneous process of rate λ^* by summing $\text{Exp}(1/\lambda^*)$ gaps and retain each candidate epoch t with probability $\lambda(t)/\lambda^*$. In an infinitesimal interval a point is produced with probability $\lambda^* dt$ and survives with probability $\lambda(t)/\lambda^*$, so the retained process has intensity $\lambda(t)$ exactly. The acceptance rate is $\approx 58\%$.

2.4 Service times by inverse-CDF sampling

Consultation times are truncated exponentials $\text{TruncExp}(\lambda_k, [a_k, b_k])$. Inverting the truncated CDF gives

$$x = -\frac{1}{\lambda} \ln \left(e^{-\lambda a} - u (e^{-\lambda a} - e^{-\lambda b}) \right), \quad u \sim U(0, 1), \quad (2)$$

which consumes *exactly one* uniform per variate. This is a deliberate choice: rejection sampling would consume a variable number and would destroy the stream synchronisation on which §2.8 depends. The realised mean after truncation follows in closed form,

$$\mathbb{E}[X] = a + \frac{1}{\lambda} - \frac{(b-a) e^{-\lambda(b-a)}}{1 - e^{-\lambda(b-a)}}, \quad (3)$$

giving 9.093 min for initial and 12.841 min for follow-up consultations. We note that a truncated exponential on $[5, 25]$ *cannot* attain

Table 2: Index values computed by the $c\mu$ and generalised $c\mu$ rules under the base paper’s parameters.

Class	T_ℓ	$\mathbb{E}[S_\ell]$	c_ℓ	$c_\ell \mu_\ell$
Level III initial	30	9.093	0.03333	0.003666
Follow-up	60	12.841	0.01667	0.001298
Level IV initial	120	9.093	0.00833	0.000916

a mean of 15 min – 15 is its supremum, approached only as $\lambda \rightarrow 0$ – so $\lambda_2 = 1/15$ is the only reading of the paper consistent with its own stated bounds, and it is the reading that reproduces the reported $\approx 85\%$ physician utilisation.

2.5 Scheduling policies

IFP, ALT and SBP are implemented directly from Equations (8)–(23) of the base paper. To these we add two policies derived from queueing theory.

The $c\mu$ rule (WSEPT). Cox and Smith [2], with the modern proof due to Buyukkoc et al. [1], show that for a multi-class queue served by a single pool under *linear* holding costs c_ℓ per unit waiting time, the policy minimising expected total holding cost always serves the non-empty class maximising the index

$$I_\ell = c_\ell \mu_\ell, \quad \mu_\ell = 1/\mathbb{E}[S_\ell]. \quad (4)$$

We set $c_\ell = 1/T_\ell$: a minute of waiting hurts in inverse proportion to how much waiting that class tolerates, which converts the triage targets directly into delay costs. The resulting indices are given in Table 2. This rule is also known as Weighted Shortest Expected Processing Time. Crucially, its indices are *constants*, so the classical rule degenerates to a fixed priority order (L3 initial > follow-up > L4 initial). That is a genuine property of the theorem, and it is exactly the limitation the generalisation removes.

The generalised $c\mu$ rule (Gc μ). For *convex* delay costs $C_\ell(w)$, Van Mieghem [15] and Mandelbaum and Stolyar [11] show the asymptotically optimal index is $C'_\ell(w) \mu_\ell$ evaluated at the head-of-line delay. Taking $C_\ell(w) = (w/T_\ell)^2$ gives

$$I_\ell(t) = \frac{2 w_{\text{head}} \mu_\ell}{T_\ell^2}, \quad (5)$$

which grows with how long the front patient has already waited. Gc μ is therefore delay-adaptive like SBP, but it is *derived* rather than tuned and has no free parameters at all.

A fair objective. The $c\mu$ rule is optimal for the holding cost, not for the unweighted mean wait, so judging it on the latter alone would be a category error. We therefore also report the *holding cost*

$$H = \frac{1}{N} \sum_{i=1}^N \frac{w_i}{T_{\ell(i)}}, \quad (6)$$

i.e. waiting measured in units of each patient’s own clinical tolerance.

2.6 Constrained simulation optimisation

The tuning problem of the base paper is

$$\min_{k_1, k_2} \bar{W}_{\text{total}}(k_1, k_2) \quad \text{s.t.} \quad SL_\ell(k_1, k_2) \geq SL_\ell^{\min}, \quad \ell \in \{3, 4\}. \quad (7)$$

Algorithm 2 Nelder–Mead iteration (minimisation)

```

1: order vertices so that  $f(x_1) \leq \dots \leq f(x_{n+1})$ 
2: stop if simplex span  $\leq x_{\text{tol}}$  and  $f(x_{n+1}) - f(x_1) \leq f_{\text{tol}}$ 
3:  $x_c \leftarrow \frac{1}{n} \sum_{i \leq n} x_i$  ▷ centroid without worst
4:  $x_r \leftarrow \Pi(x_c + \alpha(x_c - x_{n+1}))$  ▷ reflect
5: if  $f(x_r) < f(x_1)$  then
6:    $x_e \leftarrow \Pi(x_c + \gamma(x_r - x_c))$  ▷ expand
7:    $x_{n+1} \leftarrow \arg \min\{f(x_e), f(x_r)\}$ 
8: else if  $f(x_r) < f(x_n)$  then
9:    $x_{n+1} \leftarrow x_r$ 
10: else
11:    $x_k \leftarrow \Pi(x_c + \rho(\cdot - x_c))$  ▷ contract
12:   if improved then  $x_{n+1} \leftarrow x_k$ 
13:   else  $x_i \leftarrow \Pi(x_1 + \sigma(x_i - x_1)) \quad \forall i > 1$ 
14:   end if
15: end if

```

Three ingredients turn this into something a simplex method can solve.

Sample average approximation. A simulation objective is random, and simplex methods stall on noisy surfaces. We minimise instead the sample average over a *fixed* set of n patient streams [7]. Because the engine is deterministic given a stream, $\hat{F}$ is a genuine deterministic function: evaluating the same point twice returns the identical value. The common seeds also cancel between candidates, which smooths the surface far below the pointwise noise level.

Exterior penalty. Nelder–Mead is unconstrained, so the service-level constraint is folded in as

$$F(k_1, k_2) = \bar{W}_{\text{total}} + \mu \sum_{\ell \in \{3, 4\}} \max\left(0, SL_\ell^{\min} - SL_\ell\right), \quad (8)$$

with $\mu = 500$ minutes per unit of shortfall: one percentage point of service-level shortfall costs as much as five extra minutes of average waiting [13]. This is large enough that an infeasible point is never optimal, yet small enough that the penalised surface remains smooth near the boundary. Box bounds $k \geq 0$ are enforced by *projection*, so the simplex can slide along a bound rather than being repelled from it.

Nelder–Mead. We implement the downhill simplex method [12] from scratch (Algorithm 2) with the standard coefficients $\alpha = 1$, $\gamma = 2$, $\rho = 0.5$, $\sigma = 0.5$. Two details matter in practice. The initial simplex uses a *per-dimension* step (4, 15), because k_1 lives on $\approx [0, 30]$ against $T_3 = 30$ while k_2 lives on $\approx [0, 120]$ against $T_4 = 120$; a single scalar step would produce a badly conditioned simplex. Second, since the method is local, we run it from four dispersed starts and report the spread.

Here $\Pi(\cdot)$ denotes projection onto the box. The optimiser’s cost is counted in *week-long simulations*, not seconds, so the comparison against the paper’s lattice is independent of hardware and of the degree of parallelism.

2.7 Cost-based staffing

The paper varies staffing but cannot recommend a level, because it has no way to price a physician-hour against a patient-waiting-minute. We supply the missing objective:

$$C(R, k_1, k_2) = \alpha H_{\text{phys}} + \beta W_{\text{total}} + \gamma V_{\text{SLA}}, \quad (9)$$

where $R = (R_m, R_a, R_n)$ is the integer roster, H_{phys} is rostered physician-hours per week, W_{total} is total patient waiting-minutes per week and V_{SLA} counts patients past their triage target. Because the best thresholds for a 3-physician night shift are not the best for a 5-physician one, (k_1, k_2) is *re-optimised inside every candidate roster*, making this a joint rather than a sequential optimisation. The search is two-stage: all rosters in $\{2, \dots, 8\}^3$ (minus those staffing nights above days) are screened cheaply and unstable ones — where more than 15% of the week’s patients remain in system at the horizon, so the queue is growing rather than clearing — are discarded; the best candidates then receive a full Nelder–Mead re-tune.

2.8 Common random numbers and inference

For two policies A and B ,

$$\text{Var}(A - B) = \text{Var}(A) + \text{Var}(B) - 2 \text{Cov}(A, B). \quad (10)$$

Independent streams give $\text{Cov} = 0$. Under CRN both policies face the identical simulated week, so a heavy week inflates both together, Cov becomes strongly positive and the variance of the *difference* collapses while each marginal variance is unchanged [8].

Most implementations forfeit much of this benefit through desynchronisation: once two policies diverge they consume random numbers in a different order. We avoid it structurally. Six independent PCG64 substreams [14] are spawned from one seed (arrivals, triage levels, initial service, follow-up service, examination need, examination selection); the *entire* exogenous patient stream — arrival times, levels, both consultation durations, diagnostics required — is materialised *before* the run starts, so the engine draws no random number while simulating; and each variate consumes exactly one uniform by Eq. (2). Patient #1723 is therefore bit-identical under every policy.

Inference follows the base paper: paired t -tests on the per-replication means (paired, because CRN makes the design paired), a Bonferroni correction across the comparison family, and Cohen’s d in its paired form $\bar{d}/s_d$. We additionally report the smallest n satisfying $t_{0.975, n-1} s/\sqrt{n} \leq h$, solved by iteration since the t quantile itself depends on n .

3 Implementation and Experiments

3.1 Software

The simulator is implemented in Python 3.11 in approximately 7,900 lines across a 15-module library, 10 runnable experiment scripts and a 33-check verification suite. Only NumPy [4], SciPy [17], Matplotlib [6] and pandas are used; SciPy appears only for distribution quantiles, the t -distribution, and as a deliberate cross-check of our own optimiser. The library preserves the five-module architecture of the base paper’s Figure 2 so that code and paper can be read side by side: entity definition, event scheduling, resource scheduling,

Table 3: Model parameters. All values are from the base paper except the five marked [†], which it does not specify.

Parameter	Value
Horizon / replications	10,080 min (1 week) / 100
Arrival rate $\lambda_{d,h}$	Table 2, 7×24 ; peak 23.05/h
Triage mix	25% Level III, 75% Level IV
Targets T_3, T_4	30 min, 120 min
Initial consultation	TruncExp(1/9) on [5, 15]
Follow-up consultation	TruncExp(1/15) on [5, 25]
$P(\text{diagnostics})$	0.60
Modality probabilities	Lab .92, CT .55, X-ray .29, US .22
Modality durations	1.19, 2.45, 3.99, 6.58 min
Report delays	20, 30, 30, 0 min
Shifts (M/A/N)	5 / 5 / 3 physicians
$SL_{\ell}^{\min}$	0.95 [†]
$T_{\text{follow}}(c\mu)$	60 min [†]
Search box	$k_1 \in [0, 30], k_2 \in [0, 60]$ [†]
Cost weights α, β, γ	120, 0.50, 50 [†]
“At least one” exam rule	resample [†]
Base seed	20250402

strategy execution and statistical output. The complete code is available at <https://github.com/Ahnafe-Akib-Tanim/Numeric-Project-submission>.

One implementation note has a material effect. Events were initially an ordered dataclass; profiling showed Python-level comparisons in the heap consumed 22% of runtime. Replacing them with plain tuples moved comparison into C and cut per-replication time from 0.20 s to 0.076 s — which is what makes the 50,415-simulation grid search of §4.3 affordable at all.

3.2 Parameters

All parameters are taken from the base paper and are listed in Table 3. Arrival rates are its Table 2 verbatim (7×24 values). The analytic offered load implied by these parameters is

$$\rho = \frac{N(\mathbb{E}[S_1] + p_{\text{exam}}\mathbb{E}[S_2])}{\sum_s L_s R_s \cdot 7} = 0.870, \quad (11)$$

consistent with the 84.98–85.43% physician utilisation the paper reports, which confirms the parameterisation before any simulation is run.

3.3 Parameters we had to choose

The paper leaves exactly five quantities unspecified. $SL_{\ell}^{\min}$ is written as a constraint but never given a number; we use 0.95, a standard ED target, strict enough that the penalty binds. T_{follow} , needed by the $c\mu$ rule, is set to 60 min, the midpoint of T_3 and T_4 ; a sweep from 15 to 120 min leaves the policy ranking unchanged. The search box must at least contain the paper’s own reported winners at $k_1 = 26$ and $k_2 = 40$. The cost weights exist only in our extension and are stress-tested in §4.4. Finally, the paper states that 60% of patients need “at least one” test while also giving independent per-modality probabilities that jointly select none with probability 1.99%; we resample so that “at least one” holds exactly.

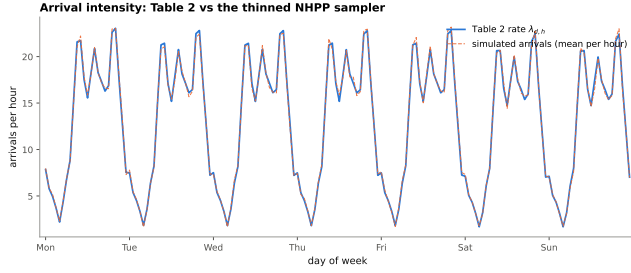


Figure 1: Validation of the thinned non-homogeneous Poisson arrival process against the base paper’s published hourly rates.

3.4 Experimental design

Every comparison uses 100 replications of one simulated week per policy under CRN, matching the base paper’s replication count. Replication r of every policy uses the identical patient stream, so all comparisons are exactly paired. Experiments were executed on a 16-core machine with 14 worker processes; the complete pipeline runs in about 20 minutes, of which the brute-force grid search is roughly half.

3.5 Verification

Correctness is checked by 33 automated assertions, each pinning down a property that some reported number depends on: the samplers against their closed forms (Eq. (3) against 200,000 draws), the day/hour indexing, convergence of the thinned arrival process, CRN seed behaviour, and engine invariants — no patient served before arrival, no idle physician while a patient waits (work conservation, verified for every policy), the sequential examination chain, and non-preemptive shift changes. The optimiser is checked against the Rosenbrock function and against SciPy. All 33 pass.

4 Results and Discussion

4.1 Model validation

Figure 1 confirms the input process: over 200 sampled weeks the realised hourly arrival counts reproduce the paper’s Table 2 with a relative L_1 error of 1.67% and correlation 0.9991.

Table 4 compares every published KPI. The agreement is structural rather than numerical. Reproduced closely: both service levels under IFP (100%/100%), ALT’s Level III waiting time (6.05 vs. 6.82 published), SBP’s Level III waiting time (12.35 vs. 10.48), physician utilisation (86.3% vs. 85.0–85.4%) and its near-invariance across policies, and — strikingly — the equipment-utilisation table (Table 5), which matches within 0.5 percentage points on all four modalities. This reproduces the paper’s central managerial conclusion that physicians, not equipment, are the bottleneck.

What does *not* reproduce is the absolute level: our waiting times run 12–25% higher. The explanation is a single number. The published parameters imply $\rho \approx 0.870$, and near saturation the mean wait is hypersensitive to the last percentage point of utilisation, so a small mismatch in the unrecoverable patient mix produces a large mismatch in waiting time. Since the underlying dataset is not public, an exact numerical match is not attainable and we do

Table 4: Validation against the base paper. Deviation is relative to the published value; 100 replications.

Policy	Metric	Ours	Paper	Dev.
IFP	Mean wait, all (min)	52.43	46.26	+13.3%
	Mean wait, Level III	50.88	44.14	+15.3%
	Service level L3 / L4	100 / 100	100 / 100	0.0%
	Physician utilisation	86.36	85.43	+1.1%
ALT	Mean wait, all (min)	42.39	37.88	+11.9%
	Mean wait, Level III	6.05	6.82	−11.2%
	Service level L3 / L4	99.96 / 87.37	98.08 / 89.29	−2.2%
	Physician utilisation	86.34	85.41	+1.1%
SBP	Mean wait, all (min)	44.08	35.25	+25.1%
	Mean wait, Level III	12.35	10.48	+17.9%
	Service level L3 / L4	99.99 / 89.40	100 / 100	−10.6%
	Physician utilisation	86.34	84.98	+1.6%

Table 5: Diagnostic equipment utilisation, reproduced from the base paper’s Table 9.

Equipment	Exams/week	Ours	Paper
Laboratory	1248	14.73%	14.36%
X-ray	394	15.59%	15.20%
CT	748	18.18%	17.64%
B-ultrasound	297	19.41%	19.03%

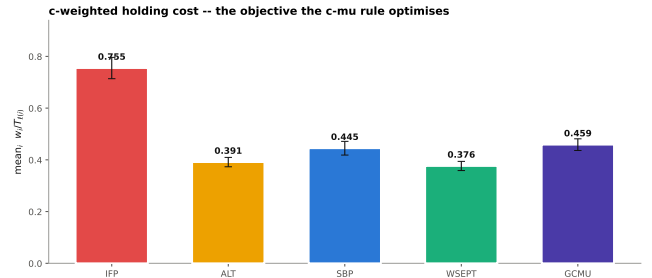


Figure 2: The objective the $c\mu$ rule is provably optimal for: waiting measured in units of each patient’s own clinical tolerance (Eq. (6)). Lower is better.

not claim one; we report the deviation openly rather than tuning parameters to hit the published figure.

4.2 Policy comparison

Table 6 is the consolidated comparison of all six policies, and Figure 3 is its geometric summary. WSEPT attains the lowest holding cost (0.3763 against IFP’s 0.7550, a 50% reduction), exactly as the Cox–Smith result predicts, and on this system it also attains the lowest unweighted mean wait (42.32 min). All differences against the paper’s three heuristics are significant after Bonferroni correction (Table 7).

The service-level columns of Table 6 carry the substantive finding: *the rules that sprint hardest on Level III are precisely the rules that let Level IV breach*. WSEPT reaches only 86.9% on Level IV, and so

Table 6: Consolidated comparison of all six scheduling policies; 100 common-random-number replications. SBP* is the Slack-Based Policy with our constrained Nelder–Mead thresholds. “Meets SLA” requires $SL_\ell \geq 95\%$ on *both* triage levels.

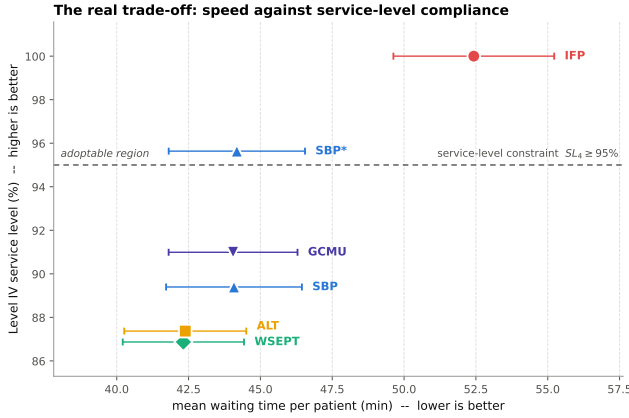
Policy	Origin	Wait (min)	95% CI	L-III	L-IV	Holding	SL_3	SL_4	Util.	Meets SLA
IFP	base paper	52.43	[49.63, 55.22]	50.88	52.94	0.7550	100.00%	100.00%	86.36%	yes
ALT	base paper	42.39	[40.26, 44.51]	6.05	54.61	0.3913	99.96%	87.37%	86.34%	no
SBP	base paper (13.1, 2.1)	44.08	[41.72, 46.44]	12.35	54.74	0.4450	99.99%	89.40%	86.34%	no
SBP*	ours (2.31, 8.37)	44.18	[41.81, 46.55]	14.14	54.27	0.4570	95.42%	95.64%	86.34%	yes
WSEPT	ours , $c\mu$	42.32	[40.21, 44.43]	3.76	55.30	0.3763	100.00%	86.87%	86.34%	no
$Gc\mu$	ours , gen. $c\mu$	44.05	[41.81, 46.29]	14.59	53.94	0.4588	100.00%	90.99%	86.35%	no

Table 7: Paired t -tests on the holding cost, WSEPT against every other policy. Bonferroni-adjusted $\alpha = 0.00333$.

Comparison	Diff.	t	p	Cohen’s d
IFP vs. WSEPT	+0.3786	31.67	1.3×10^{-53}	3.17
ALT vs. WSEPT	+0.0150	36.54	2.9×10^{-59}	3.65
SBP vs. WSEPT	+0.0686	14.15	1.6×10^{-25}	1.42
WSEPT vs. $Gc\mu$	-0.0824	-33.42	1.0×10^{-55}	-3.34

Table 8: Simulation budget required by brute-force grid search and by the hand-implemented Nelder–Mead simplex, on the identical objective and the identical patient streams.

Method	Points	Simulations	Objective
Grid, coarse (step 1.0)	1,891	28,365	47.6105
Grid, fine (step 0.1)	441	22,050	46.4601
Grid, total	2,332	50,415	46.4601
Nelder–Mead, 1 start	53	2,650	46.4668
Nelder–Mead, 4 starts	196	9,800	46.4668

**Figure 3: Speed against service-level compliance. Only policies above the dashed constraint line are adoptable; of the tuned policies only SBP* is both fast and feasible.**

do ALT and, importantly, the paper’s own tuned SBP at 89.4%. Only IFP — which is slow — and our re-tuned SBP* satisfy the constraint on both levels. Consequently there is no single winner, and the honest answer depends on the objective: WSEPT minimises both the holding cost and the raw mean wait; SBP* is the fastest policy a hospital could actually adopt.

We state one caveat explicitly. The Cox–Smith optimality proof assumes a single-stage multi-class queue; our model has re-entrant flow, since patients return for a follow-up after diagnostics, and a time-varying server pool. The optimality guarantee is therefore approximate here, and we report the empirical ranking rather than relying on the theorem.

4.3 Optimiser versus brute force

Table 8 is the central efficiency result. Reproducing the paper’s procedure on our model, the two-stage lattice evaluated 2,332 points at a cost of 50,415 week-long simulations — roughly 970 simulated patient-years of ED operation — to reach (4.10, 8.20). Our simplex reached an equally good point in 2,650 simulations, a 19.0× reduction. Re-scored afterwards on one identical full-length batch, the two optima differ by 0.006 minutes, far below the simulation’s own noise floor.

Four dispersed starts converge to objectives within 0.15 min of one another (46.468–46.622), indicating an effectively unimodal penalised surface in this box, and our implementation agrees with `scipy.optimize` to 0.0000 min on the same objective. Figure 4 draws the simplex trajectory on the very response surface the lattice had to evaluate exhaustively; Figure 5 shows solution quality against budget.

This experiment also produced an unanticipated finding. Under the 95% constraint the optimum moves to a much larger k_2 , and the paper’s published (13.1, 2.1) becomes *infeasible* on our model, reaching only 89.4% on Level IV (Table 9). The mechanism is transparent: with $T_4 = 120$ and $k_2 = 2.1$, a Level IV patient is flagged urgent only after waiting 117.9 of their 120 permitted minutes, which is too late to rescue them on a system as congested as ours. This is the strongest argument in this work for re-tuning under an explicit constraint rather than reusing published parameters.

4.4 Cost-optimal staffing

Table 10 prices the paper’s own seven staffing scenarios under Eq. (9). Where the paper can only observe that more physicians means shorter waits — a statement with no stopping point — the priced sequence has a clear interior optimum at S_5 . A free search over all 140 admissible rosters, with (k_1, k_2) re-tuned inside each,

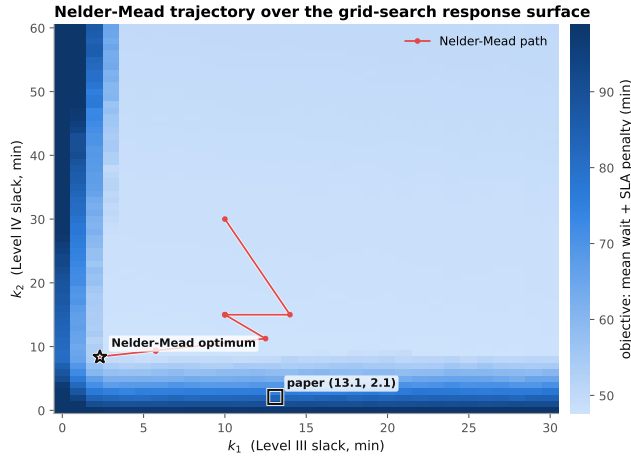


Figure 4: The Nelder–Mead trajectory over the grid-search response surface. The dark band is the service-level penalty region; the base paper’s published optimum (square) lies inside it.

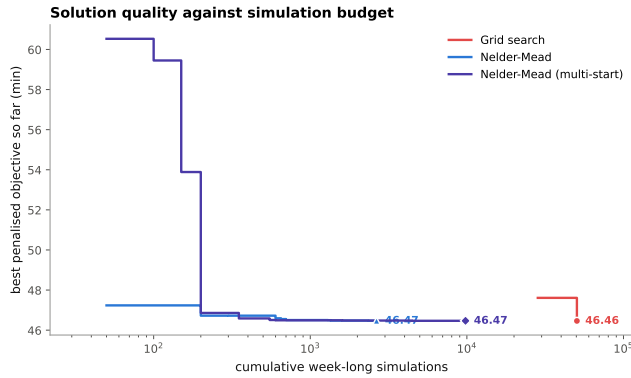


Figure 5: Best objective found so far against cumulative simulation budget (log scale).

Table 9: All three optima re-scored on identical replications.

Setting	(k_1, k_2)	Wait	SL_3	SL_4	Status
Nelder–Mead	(2.31, 8.37)	44.18	95.42%	95.64%	feasible
Grid search	(4.10, 8.20)	44.17	97.73%	95.49%	feasible
Paper	(13.10, 2.10)	44.08	99.99%	89.40%	infeasible

finds a cheaper configuration still: roster (6, 6, 3) with thresholds (12.20, 23.40). Relative to the paper’s fixed (5, 5, 3) this costs 105 additional physician-hours per week but recovers 33 minutes of mean waiting per patient and cuts the SLA penalty from 4,913 to 60 cost units, for a net weekly saving of 28,878 units, or 20.7%.

The joint formulation earns its complexity. Screened at one fixed threshold pair, roster (6, 6, 3) scores 111,932; re-tuned to its own optimum (12.20, 23.40) it scores 110,870. More importantly the thresholds *move substantially* between rosters — k_2 ranges from

Table 10: The base paper’s seven staffing scenarios, priced under Eq. (9). Cost in arbitrary units per week.

	Roster	Phys-h	Wait	SL_4	Total
S_0	(5,5,3)	714	44.18	95.64%	139,391
S_1	(5,5,4)	777	35.19	98.53%	134,121
S_2	(5,5,5)	840	31.25	99.32%	136,490
S_3	(5,6,5)	889	16.34	99.98%	124,942
S_4	(5,7,5)	938	11.45	100.00%	125,327
S_5	(6,7,5)	994	4.32	100.00%	124,082
S_6	(7,7,5)	1050	2.38	100.00%	128,652
Ours	(6,6,3)	819	10.97	99.93%	110,513

Table 11: Sensitivity of the staffing recommendation to the cost weights.

Scenario	α	β	γ	Winner	vs. (5,5,3)
Baseline	120	0.500	50	(6,6,3)	−20.7%
Waiting $\times 4$	120	2.000	50	(6,7,3)	−53.2%
Waiting $\times \frac{1}{4}$	120	0.125	50	(5,6,3)	−5.2%
Physicians $\times 1.5$	180	0.500	50	(6,6,3)	−12.4%
SLA breach $\times 10$	120	0.500	500	(6,6,3)	−39.5%

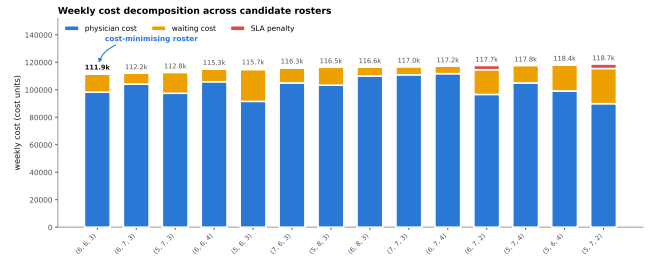


Figure 6: Weekly cost decomposed into physician, waiting and SLA-penalty components across the best screened rosters.

11.5 to 25.6 across the six refined candidates — so holding them fixed while varying staffing would score every alternative roster at settings chosen for a different one.

Table 11 stress-tests the recommendation. The winning roster *changes* with the weights. We regard this not as a weakness of the method but as the finding itself: the right staffing level genuinely depends on how a hospital prices a patient’s waiting minute against a physician’s salary, so those weights must be agreed before a roster can be set. What does not change is that every weighting beats the paper’s fixed (5, 5, 3), by between 5.2% and 53.2%.

4.5 Variance reduction

Table 12 reports the effect of CRN on every pairwise comparison. The induced correlations range from +0.996 to +0.9996, against ≈ 0 under independent streams, and the median variance removed is 98.95%, a median efficiency gain of 95.8%. Confidence intervals on the mean difference narrow by factors of 4 to 40.

Table 12: Variance of the pairwise difference in mean waiting time, with and without common random numbers; 100 replications per arm.

Comparison	Var ind.	Var CRN	Removed	Gain
IFP vs. ALT	241.34	12.44	94.85%	19.4×
IFP vs. SBP	330.48	6.12	98.15%	54.0×
IFP vs. WSEPT	288.20	13.16	95.43%	21.9×
IFP vs. Gcμ	216.19	8.50	96.07%	25.4×
ALT vs. SBP	220.77	2.15	99.03%	102.9×
ALT vs. WSEPT	191.64	0.11	99.94%	1789.3×
ALT vs. Gcμ	223.75	0.46	99.79%	485.4×
SBP vs. WSEPT	198.04	2.23	98.87%	88.7×
SBP vs. Gcμ	268.97	1.03	99.62%	261.8×
WSEPT vs. Gcμ	215.27	0.59	99.72%	362.7×
Median			98.95%	95.8×

What Common Random Numbers buy (target CI half-width 0.5 min)

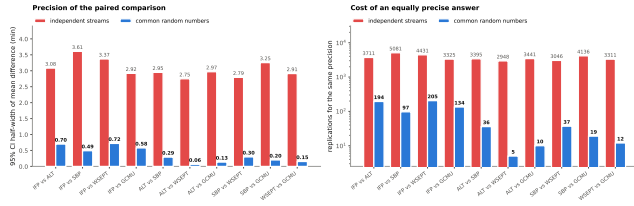


Figure 7: Left: 95% confidence-interval half-width on each pairwise difference. Right: replications needed for ± 0.5 min precision (log scale).

Two properties deserve emphasis. First, CRN is not a bias: the per-policy means and standard deviations agree between the two arms within sampling error, as theory requires, since only the joint distribution changes. Second, it changes conclusions — several comparisons that fail to reach the Bonferroni threshold under independent streams are decisive under CRN. Where the arms disagree, CRN is the arm to believe, because it estimates the same quantity with lower variance.

We are careful not to over-claim. A 95.8× efficiency gain does not mean one replication suffices: the ratio is itself estimated, and a t -interval needs adequate degrees of freedom. The defensible practical reading is that 20–30 CRN replications give comparisons at least as sharp as the base paper’s 100 independent ones — a 3–5× saving that can safely be taken.

4.6 Robustness

We reproduce all three of the base paper’s sensitivity studies and extend them to the added policies. Scaling every arrival rate by -15% to $+15\%$ (Figure 8) drives ρ from 0.739 to 1.000; the policies are nearly interchangeable under light load and diverge sharply as the system saturates, the policy spread growing from 1.6 to 53 min. Across the seven staffing scenarios (Figure 9) waits fall with clear diminishing returns and the choice of policy matters most exactly when physicians are scarce, the spread falling from 10.1 min at S_0 to 0.06 min at S_6 . Finally, replacing the truncated exponential service

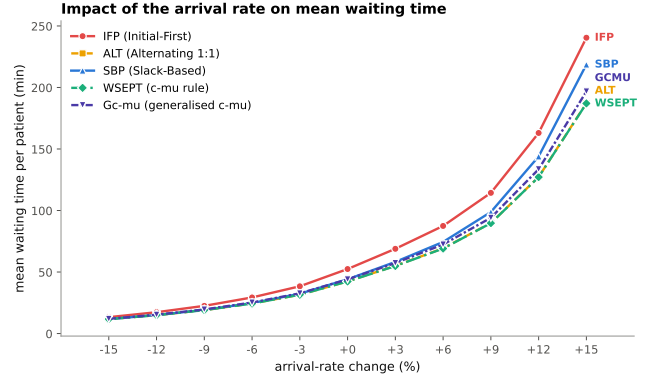


Figure 8: Impact of the arrival rate on mean waiting time, all five policies.

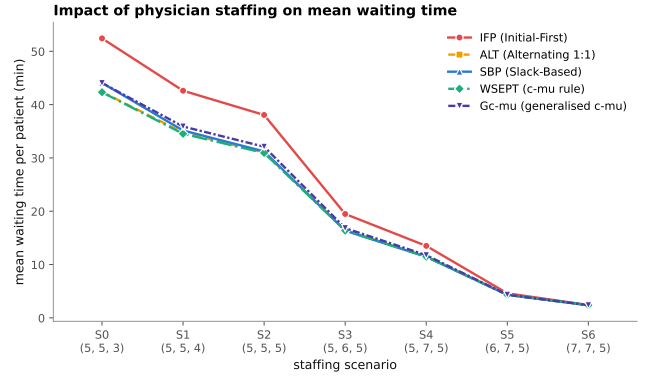


Figure 9: Impact of physician staffing on mean waiting time across the base paper’s seven scenarios.

times with truncated lognormals of the same nominal means raises all waits by 44–54% through the heavier right tail, but leaves the ranking *identical*, confirming that the comparative conclusions are not an artefact of the distributional assumption. All three findings match the base paper.

4.7 Threats to validity

Three limitations bound the claims above. *Construct*: the base paper’s dataset is unavailable, so we calibrate to published summary statistics only; the 12–25% level mismatch is the visible consequence, and only relative conclusions should be carried forward. *Internal*: five parameters are our choice; four are swept or stress-tested, and the fifth ($SL^{\min}$) is reported alongside both raw service levels so a different threshold can be applied by inspection. *External*: the model is one ED with one triage system; the $c\mu$ optimality guarantee is weakened by re-entrant flow, and the cost weights are illustrative rather than institutional.

5 Conclusion and Future Work

We rebuilt the discrete-event model of Lv et al. [10] from its published parameters alone and validated it against every KPI the

paper reports. All six structural findings reproduce, including the equipment-utilisation table to within 0.5 percentage points and all three sensitivity-analysis conclusions; absolute waiting times run 12–25% higher, which we trace to the implied load $\rho \approx 0.87$ and report rather than tune away.

Four extensions follow. A $c\mu$ -based policy, provably optimal for a stated delay-cost objective, attains the lowest holding cost (–50% against IFP) and the lowest raw mean wait. A hand-implemented Nelder–Mead simplex with an exterior penalty reaches the lattice optimum to within 0.006 min using 19× fewer week-long simulations. An explicit cost model identifies roster (6, 6, 3) with re-tuned thresholds as 20.7% cheaper per week than the paper’s fixed (5, 5, 3), with the winner’s sensitivity to the cost weights reported as a finding in its own right. Exact common random numbers remove 98.9% of the variance of every pairwise comparison.

The single most consequential result is negative: on our model the base paper’s own published thresholds (13.1, 2.1) violate a 95% Level IV service-level constraint, because $k_2 = 2.1$ defers intervention to minute 117.9 of 120. Constrained re-tuning restores feasibility and yields the only tuned policy that a hospital could adopt. Published scheduling parameters are properties of the system they were fitted to, not transferable constants.

Future work follows three lines. The $c\mu$ guarantee should be re-derived for the re-entrant, time-varying setting actually present here, where only asymptotic results are currently available. The optimiser could be extended to higher-dimensional decision vectors — per-shift thresholds, or thresholds that adapt to the observed queue state — where the advantage over lattice search grows combinatorially and where the base paper itself suggests reinforcement learning. Finally, the cost model invites a formal stochastic-programming treatment in which the roster is a first-stage decision and the thresholds a recourse decision, which would replace our two-stage screening heuristic with a principled decomposition.

Acknowledgments

We thank the course instructors of CSE 402 for guidance on scope and methodology. All simulation code, generated result tables, figures and a complete provenance log recording the exact command behind every artefact are available at <https://github.com/Ahnaif-Akib-Tanim/Numeric-Project-submission>.

References

- [1] Cagatay Buyukkoc, Pravin Varaiya, and Jean Walrand. 1985. The $c\mu$ Rule Revisited. *Advances in Applied Probability* 17, 1 (1985), 237–238. doi:10.2307/1427064
- [2] David R. Cox and Walter L. Smith. 1961. *Queues*. Methuen, London, UK.
- [3] Andrew J. Davenport, Christophe Gefflot, and J. Christopher Beck. 2001. Slack-Based Techniques for Robust Schedules. *Proceedings of the Sixth European Conference on Planning (ECP-2001)* (2001), 7–18.
- [4] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, et al. 2020. Array Programming with NumPy. *Nature* 585, 7825 (2020), 357–362. doi:10.1038/s41586-020-2649-2
- [5] Junfei Huang, Boaz Carmeli, and Avishai Mandelbaum. 2015. Control of Patient Flow in Emergency Departments, or Multiclass Queues with Deadlines and Feedback. *Operations Research* 63, 4 (2015), 892–908. doi:10.1287/opre.2015.1389
- [6] John D. Hunter. 2007. Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering* 9, 3 (2007), 90–95. doi:10.1109/MCSE.2007.55
- [7] Sujin Kim, Raghu Pasupathy, and Shane G. Henderson. 2015. A Guide to Sample Average Approximation. *Handbook of Simulation Optimization* (2015), 207–243. doi:10.1007/978-1-4939-1384-8_8
- [8] Averill M. Law. 2015. *Simulation Modeling and Analysis* (5th ed.). McGraw-Hill Education, New York, NY, USA.
- [9] Peter A. W. Lewis and Gerald S. Shedler. 1979. Simulation of Nonhomogeneous Poisson Processes by Thinning. *Naval Research Logistics Quarterly* 26, 3 (1979), 403–413. doi:10.1002/nav.3800260304
- [10] Wei Lv, Runzhang Liu, Feiyi Yan, and Yan Wang. 2026. Discrete Event Simulation-Based Analysis and Optimization of Emergency Patient Scheduling Strategies. *Healthcare* 14, 1 (2026), 99. doi:10.3390/healthcare14010099
- [11] Avishai Mandelbaum and Alexander L. Stolyar. 2004. Scheduling Flexible Servers with Convex Delay Costs: Heavy-Traffic Optimality of the Generalized $c\mu$ -Rule. *Operations Research* 52, 6 (2004), 836–855. doi:10.1287/opre.1040.0152
- [12] John A. Nelder and Roger Mead. 1965. A Simplex Method for Function Minimization. *Comput. J.* 7, 4 (1965), 308–313. doi:10.1093/comjnl/7.4.308
- [13] Jorge Nocedal and Stephen J. Wright. 2006. *Numerical Optimization* (2nd ed.). Springer, New York, NY, USA. doi:10.1007/978-0-387-40065-5
- [14] Melissa E. O’Neill. 2014. PCG: A Family of Simple Fast Space-Efficient Statistically Good Algorithms for Random Number Generation. *ACM Trans. Math. Software* (2014). Technical Report HMC-CS-2014-0905, Harvey Mudd College.
- [15] Jan A. Van Mieghem. 1995. Dynamic Scheduling with Convex Delay Costs: The Generalized $c\mu$ Rule. *The Annals of Applied Probability* 5, 3 (1995), 809–833. doi:10.1214/aop/1177004706
- [16] Lien Vanbrabant, Kris Braekers, Katrien Ramaekers, and Inneke Van Nieuwenhuyse. 2019. Simulation of Emergency Department Operations: A Comprehensive Review of KPIs and Operational Improvements. *Computers & Industrial Engineering* 131 (2019), 356–381. doi:10.1016/j.cie.2019.03.025
- [17] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, et al. 2020. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods* 17, 3 (2020), 261–272. doi:10.1038/s41592-019-0686-2
- [18] Lien Wang and Erik Demeulemeester. 2023. Simulation Optimization in Healthcare Resource Planning: A Literature Review. *IIE Transactions* 55, 10 (2023), 985–1007. doi:10.1080/24725854.2022.2147606